TIMC 返修：随机扰动实验（实验 B）实施说明

交付对象：ChatGPT（可直接照此实现并运行）　　代码基线：mikealla-w/TIMC，分支 revision

1 这份文档要解决什么

返修方案第 7 节要求补一组“非周期随机扰动”实验，用于回答审稿人关于鲁棒性的问题。当前仿真中的扰动是 4 个正弦叠加的确定性信号（见 load_lifting_payload_dynamics.m），只能覆盖周期性扰动；审稿人 1（R1-6）与审稿人 2（R2-1）都要求证据覆盖更一般的扰动形式。

本实验的目标不是“证明复合方法更好”，而是一个可回答的问题：

核心问题：在固定随机种子、统一参数集的非周期扰动下，四种控制器的排序是否与周期扰动工况一致？如果不一致，差异出现在哪一类指标上？

据此，实验只增加扰动模型与批处理脚本，不改控制器、不改指标定义、不重新调参。

2 代码现状（实现前必须了解）

2.1 仓库与分支

仓库 github.com/mikealla-w/TIMC，工作分支 revision（main 为原始快照，禁止改动）。所有改动提交到 revision，不新建分支，不推送 main。

2.2 单次运行入口

```
r = run_load_lifting_case(controller_type, with_disturbance, options);
```

controller_type 取 'backstepping' 或 'rbf'；with_disturbance 为逻辑值；options 支持以下字段：

字段	含义	默认值
controller_param_override	递归合并到 controller_config 的覆盖结构体	[]
uav_controller_type	无人机内环控制器类型	'backstepping'
save_run_data	是否保存 run_result.mat	true
save_workspace	是否保存完整工作区（批量时务必 false）	false
run_output_dir	结果目录	pwd/simulation_runs

enable_default_plots	是否绘制默认图	false
dt / Tf	步长与时长覆盖	[]（用默认 1 ms / 20 s）

2.3 控制器的三个开关

四组消融通过 controller_param_override 中的三个布尔量切换，公共参数保持不变：

```
override.rbf.base_mode            = 'backstepping';
override.rbf.enable_compensation = true/false;   % RBF 补偿
override.rbf.enable_adaptation   = true/false;   % RBF 自适应
override.eso.use_compensation    = true/false;   % ESO 补偿
```

2.4 现有扰动实现

load_lifting_payload_dynamics.m 在 with_disturbance=true 时叠加：

```
d_R = 1*sin(t) + 0.2*sin(2t) + 0.3*sin(3t) + 0.4*sin(4t);   % rad/s^2
d_p = 3*sin(t) + 0.2*sin(2t) + 0.3*sin(3t) + 0.4*sin(4t);   % m/s^2
```

d_R 是三轴共用的机体角加速度，d_p 是三轴共用的世界系加速度；它们分别加到姿态与平移通道。新随机扰动应叠加在同样的位置。

2.5 可直接复用的范式

上一步为“未建模阻力”增加的 payload_drag_cd 提供了一个可照抄的注入范式：主脚本从 experiment.payload_plant.drag_cd 读取参数，写入同名 global，动力学函数用 global 读取。随机扰动应沿用这一模式，避免修改函数签名。

2.6 指标函数

```
m = compute_revision_metrics(run_result);
```

字段	含义
load_pos_rms / load_pos_max	位置 RMS 与最大误差（最后 20% 时间窗）
rot_err_rms / rot_err_max	旋转角误差（arccos 形式）
tension_min_steady	约化模型最小有符号张力（N）
tension_negative_samples	负张力样本数
alloc_force_residual_rms	分配力残差 RMS

3 需要新增的三样东西

3.1 随机扰动生成器

新建 make_bandlimited_disturbance.m，签名如下：

```
[trace] = make_bandlimited_disturbance(t, seed, cfg)
```

输入：t 为仿真时间列向量（0:dt:Tf），seed 为整数，cfg 含 bandwidth_hz、amp_pos、amp_att。输出 trace 为结构体，字段 t（列向量）、a_p（3×N）、a_R（3×N）。

生成方法（一阶低通白噪声，即 AR(1)）：

```
rng(seed, 'twister');
a = exp(-2*pi*fc*dt);                % fc = cfg.bandwidth_hz
w = randn(3, N);                     % 每个通道独立
x(:,1) = w(:,1);
for k = 2:N
    x(:,k) = a*x(:,k-1) + sqrt(1-a^2)*w(:,k);
end
x = x - mean(x,2);                   % 去均值
x = x ./ sqrt(mean(x.^2,2)) * amp;   % 归一化到目标 RMS
```

注意：randn 是白噪声，直接叠加会在 1 kHz 采样下产生过大的高频能量，必须经过低通并归一化；三个平移通道与三个姿态通道分别独立生成。

3.2 注入方式

- **配置：**在 load_lifting_controller_params.m 或 experiment 配置中增加 experiment.disturbance.random 结构体，字段见下表。

字段	含义	建议值
enabled	是否启用随机扰动	true
seed	随机种子	101 / 202 / 303 / 404 / 505
amp_pos	平移三轴目标 RMS（m/s²）	1.0
amp_att	姿态三轴目标 RMS（rad/s²）	0.3
bandwidth_hz	低通截止频率	1.0

- **生成：**主脚本 load_lifting_main.m 在参数加载后调用生成器，把结果写入 global payload_random_disturbance；当 enabled=false 时写入空值。

- **注入**：load_lifting_payload_dynamics.m 在 with_disturbance 分支中叠加随机项，用 interp1(trace.t, trace.a_p(i,:), t, 'linear', 0) 取当前时刻值（RK4 的中间时刻也能正确取值）。关闭时该项为 0，行为必须与当前代码完全一致。
- **互斥**：随机扰动与周期扰动是二选一还是叠加，必须明确并在代码里写死：建议“启用随机扰动时不再叠加周期扰动”，避免两次计入外部扰动。

3.3 批处理脚本

新建 run_revision_random_cases.m，结构照抄 run_revision_drag_cases.m：

```
methods = { 'BS'          (backstepping, 无开关)
            'BS_RBF'      (rbf, comp=true,  adapt=true,  eso=false)
            'BS_ESO'      (rbf, comp=false, adapt=false, eso=true)
            'BS_RBF_ESO'  (rbf, comp=true,  adapt=true,  eso=true) }
seeds   = [101 202 303 404 505]
```

推荐矩阵 4 方法 × 5 种子 = 20 次（约 70–80 分钟）。若时间受限，可去掉 BS_RBF 得到 15 次，但必须在文中说明少一组的原因。

4 扰动参数取值与依据

项目	取值	依据
平移 RMS	1.0 m/s²	约为重力加速度的 10%，与现有周期扰动同量级
姿态 RMS	0.3 rad/s²	与现有 d_R 的有效幅值相当
带宽	1 Hz	低于 ESO 带宽（ω=50 → 约 8 Hz），保证观测器有事可做但不至于失效
种子	101/202/303/404/505	固定列表，预试验与正式试验分开
时长与步长	20 s / 1 ms	与已有实验一致，便于横向比较

预试验用 101 检查量级是否合理（不发散、张力不为负）；确认后再跑全部种子。如果 101 下基础反步直接发散，应把 amp_pos 降到 0.5 再重试，并在文档中记录这一调整。

5 指标、统计与失败判据

每个 (方法, 种子) 记录以下字段，汇总为 CSV：

列名	说明
method	BS / BS_RBF / BS_ESO / BS_RBF_ESO
seed	随机种子
status	ok 或 failed: 原因
load_pos_rms	位置 RMS（m，最后 20% 时间窗）
load_pos_max	最大位置误差（m）
rot_err_rms	旋转角误差 RMS（rad）
tension_min	最小有符号张力（N）
trace_hash	扰动轨迹的 SHA-256 前 16 位，用于复核

按方法聚合时给出 mean ± std 与失败数；5 个种子只能作为初步离散度，正文不得声称统计显著性。

失败判据：出现 NaN/Inf、位置误差超过 5 m、或仿真未跑完。失败必须记录并计入失败数，不得删除或事后截断。

6 输出规范

```
revision_runs/random/<method>_s<seed>/
    simulation_run_*.mat        % run_result（save_workspace 必须为 false）
    revision_metrics.mat
    disturbance_trace.mat       % 该种子对应的 trace
revision_runs/random/revision_random_summary.csv
revision_runs/random/revision_random_summary.md    % 人读版表格
```

7 验收标准（交付前逐条自查）

编号	检查项	通过标准
V1	可复现性	同一 seed 连续跑两次，trace_hash 完全一致
V2	回归安全	enabled=false 时，结果与已有周期扰动实验逐位一致
V3	完整性	20 次运行全部产出 status 字段，无缺失
V4	物理合理性	张力记录非空；若出现负张力，

		如实记录而非裁剪
V5	口径一致	指标、时间窗、参数集与已有实验完全相同
V6	仓库卫生	revision_runs/ 已被 .gitignore 忽略，提交中不含 MAT 文件

8 常见坑（按踩坑概率排序）

- **rng 状态污染：**每个 case 开始前必须 rng(seed,'twister')；不要依赖上一轮的全局状态，也不要在一个 case 内多次重置。
- **白噪声直接叠加：**1 kHz 采样下 randn 的能量集中在高频，会把分配器和内环打崩。必须低通 + 归一化，并检查首批结果的张力与误差量级。
- **时间网格不一致：**trace 必须定义在与仿真相同的 t 上；RK4 的中间时刻用 interp1 取值，不能用 ZOH 近似，否则等效能量不同。
- **叠加两遍扰动：**随机扰动与周期扰动必须互斥，否则总扰动翻倍，结果无法与已有实验比较。
- **复用旧缓存：**旧 MAT 来自确定性扰动，任何缓存复用都会污染结论；批量脚本必须强制重跑。
- **并行与随机数：**若用 parfor，每个 worker 需独立 rng(seed+worker_offset)，且种子必须可追溯；更稳妥的做法是串行跑。
- **忘记关闭阻力：**payload_drag_cd 必须为 0，否则混入上一实验的未建模项。
- **改了控制器：**本实验只加扰动。任何对 RBF/ESO 参数的调整都会破坏“统一参数集”的公平性。

9 可直接粘贴给 ChatGPT 的任务描述

以下整段可原样发给 ChatGPT（它已能访问该仓库）：

> 你在仓库 `github.com/mikealla-w/TIMC` 的 `revision` 分支上工作，不要改 `main`。
>
> 任务：补一组“非周期随机扰动”实验，比较四种控制器：`BS`、`BS+RBF`、`BS+ESO`、`BS+RBF+ESO`。
>
> 要求：
>
> 1. 新增 `make_bandlimited_disturbance.m`：输入 `(t, seed, cfg)`，输出 `trace`（`t, a_p 3xN, a_R 3xN`）。
>
> 用 `AR(1)` 低通白噪声：`a=exp(-2*pi*fc*dt)`，`x_k=a*x_{k-1}+sqrt(1-a^2)*w_k`，

去均值后归一化到目标 RMS。fc=1 Hz，平移 RMS=1.0 m/s^2，姿态 RMS=0.3 rad/s^2。

每个 case 前 rng(seed,'twister')。

2. 在 load_lifting_main.m 中根据 experiment.disturbance.random 生成 trace 并写入 global payload_random_disturbance；在 load_lifting_payload_dynamics.m 中叠加，用 interp1 取任意时刻值。启用随机扰动时不再叠加原有周期扰动。

   enabled=false 时结果必须与当前代码完全一致。

3. 新增 run_revision_random_cases.m，跑 4 方法 x 5 种子（101/202/303/404/505），结果写 revision_runs/random/<method>_s<seed>/，汇总写 revision_random_summary.csv，

   列：method, seed, status, load_pos_rms, load_pos_max, rot_err_rms, tension_min, trace_hash。

   指标统一用 compute_revision_metrics。save_workspace 必须为 false。

4. 验收：同 seed 两次运行 trace_hash 一致；20 次全部完成；不改控制器参数与指标定义；不提交 MAT 文件与 revision_runs。

5. 完成后报告：汇总表、失败项、以及“随机扰动下四种方法的排序是否与周期扰动一致”的结论。

注意：每个 case 独立初始化（含 rng）；不要复用旧缓存；payload_drag_cd 必须为 0。

10 工作量与排期

环节	估时
生成器 + 注入 + 回归检查	1.5–2 小时
批处理脚本 + 单案例调试	1 小时
20 次运行	约 1.5 小时（单次约 4 分钟）
汇总与报告	0.5 小时
合计	约 4.5–5 小时

附：与已有实验的衔接

随机扰动实验的结论必须与已完成的三个实验放在一起解读：

- 四组消融（周期扰动）：ESO 主导，复合比 ESO-only 好约 10%。
- 参数失配 0.5–1.5 倍：复合/ESO 比值恒为 0.903，说明 RBF 不具备失配特异性。
- 未建模二次阻力（最高约 74% 重量）：比值 0.903→0.907，RBF 无法表达速度相关项。

因此随机扰动实验最可能的结果是“排序不变、比值仍在 0.90 附近”。如果出现与此不同的结果（例如某方法在随机扰动下明显变差），必须如实报告并解释，不能为了与已有结论一致而调参。